

Transformers for Language Models

In previous NLP coursework, we focused on representing text in numerical form so it could be processed by machine learning models. Methods such as Bag of Words (BoW) and TF-IDF capture word usage patterns within documents and make text amenable to standard learning algorithms.

These representations are effective when word presence or frequency is sufficient. However, they treat text as an unordered collection of words and do not account for how meaning depends on word order or context.

To address this, word embeddings were introduced as a more expressive representation. By mapping words into continuous vector spaces, embeddings capture semantic similarity and relationships that frequency-based methods cannot. Each word is assigned a single, fixed vector, regardless of how it is used across different contexts.

This limitation motivates a shift from representing individual words to modelling language as a sequence. Meaning must be derived from how words interact within a sentence, rather than from words in isolation.

So, we will discuss encoder-decoder architectures, followed by attention mechanisms, which allow models to focus dynamically on relevant parts of a sequence. Only after these ideas are established, we will introduce Transformers, which retain attention while removing recurrence entirely.

Although the module is centred on Transformer-based language models, it deliberately begins with the concepts that lead to them. Transformers are studied as the outcome of a progression, from word-level representations to context-aware, sequence-based language modelling.

We will now discuss word embeddings in detail.

Word Embeddings

Word embeddings are numerical representations of words in a continuous vector space. Each word is mapped to a fixed-length vector of real numbers, typically learnt automatically from large text corpora during training.

Unlike rule-based representations, word embeddings allow machine learning models to work directly with numerical inputs while preserving semantic relationships between words.

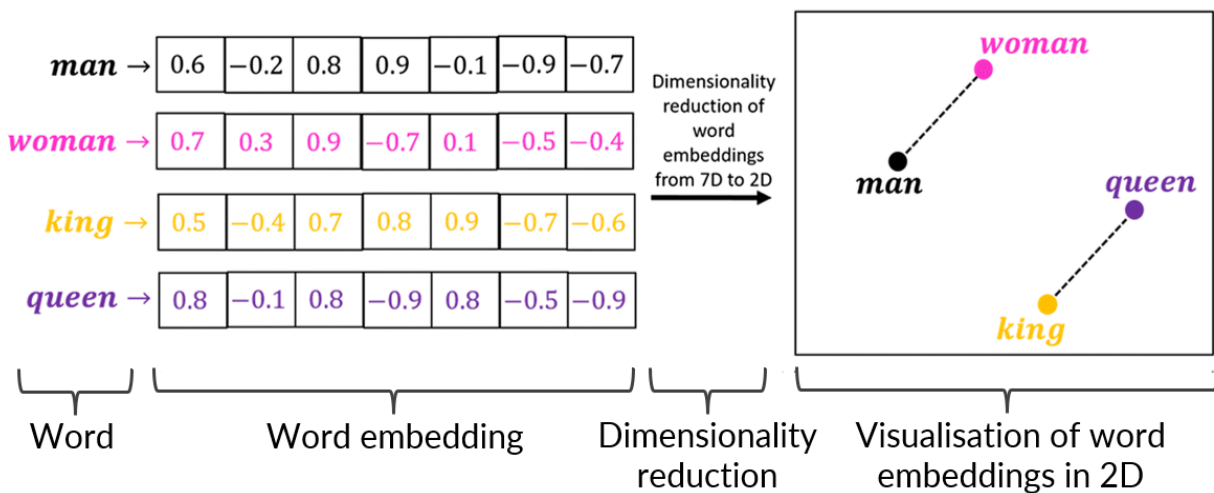
A core assumption of word embeddings is that **words occurring in similar linguistic contexts tend to have similar meanings** and, therefore, should have similar vector representations.

In word embeddings, semantic meaning is distributed across all dimensions of the vector. No single dimension corresponds to a specific interpretable concept such as sentiment, tense or category.

Instead, meaning emerges from the following:

- Relative distances between word vectors
- Directions in the embedding space
- Clusters formed by words with similar usage patterns

Consequently, semantic similarity is captured geometrically rather than symbolically, as shown below.



Word embeddings usually exist in high-dimensional spaces (often 100–300 dimensions), which are not directly interpretable. To analyse them visually, dimensionality reduction techniques are used to project embeddings into 2 or 3 dimensions.

Such visualizations:

- Help build intuition about semantic similarity
- Reveal clusters of related words
- Are used only for inspection and analysis

The original embeddings used by the model remain unchanged.

Different methods can be used to generate word embeddings such as Word2Vec, GloVe and FastText.

Word2Vec

Word2Vec is a neural network–based method for learning word embeddings using local context information. It learns word representations by predicting words based on their neighboring words in a sentence.

The method relies on the idea that local context captures semantic meaning and that words appearing in similar neighborhoods share semantic properties.

Word2Vec provides two training architectures: Continuous Bag of Words and Skip-Gram.

Continuous Bag of Words (CBOW)

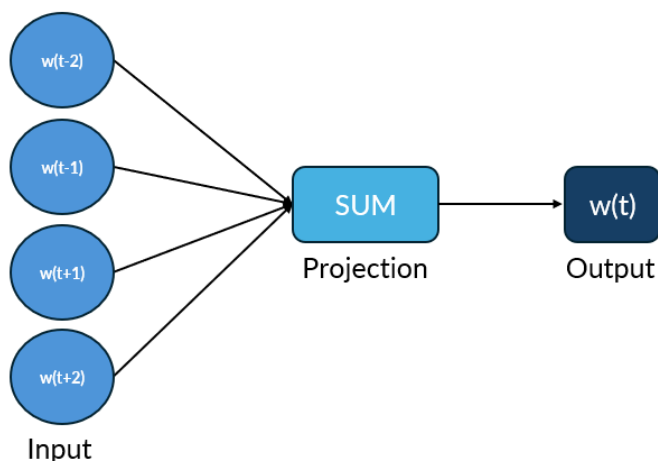
The Continuous Bag of Words model predicts a target word from its surrounding context words.

The process followed is as follows:

- A fixed-size context window is selected around a target word.
- Each context word is mapped to an embedding vector.
- Context vectors are combined into a single representation, typically by averaging.
- The model predicts the target word from this combined context.

Training samples are constructed as (*context window*, *target word*) pairs.

CBOW is computationally efficient and performs well when the data set contains many frequent words.



Skip-Gram

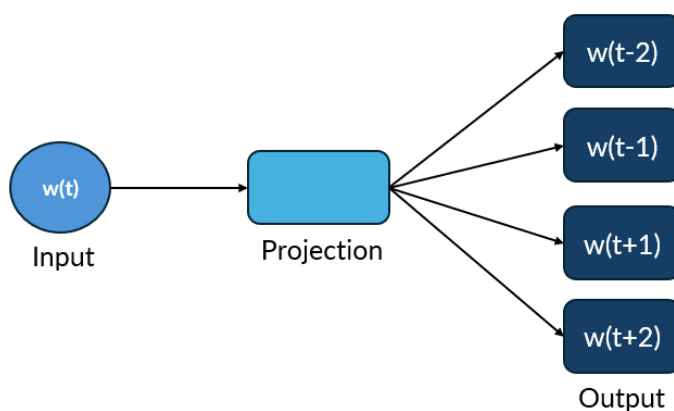
The Skip-Gram model reverses CBOW's prediction objective. Instead of predicting a target word from the context, it predicts **surrounding context words** given a single target word.

The process followed is as follows:

- A target word is selected.
- Context words within a fixed window are identified.
- Each context word is predicted independently.

Training samples are constructed as *(target word, context word)* pairs.

Skip-gram generates multiple training signals for each word occurrence and is particularly effective for learning representations of rare words.



CBOW and Skip-gram differ in terms of how they model word-context relationships.

- CBOW predicts the target word using surrounding context words.
- Skip-gram predicts surrounding context words using the target word.

CBOW is faster to train and is suitable for frequent vocabulary. Skip-Gram is computationally more expensive but captures richer representations for infrequent words.

Global Vectors (GloVe)

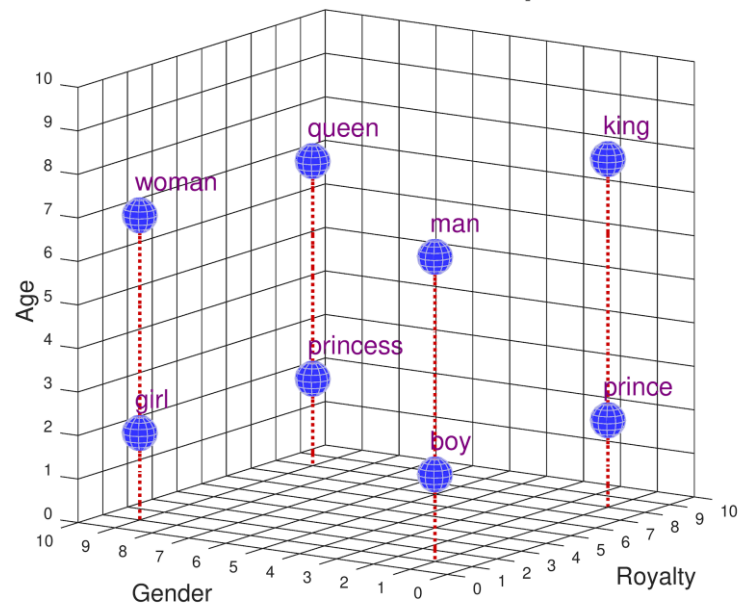
GloVe learns word embeddings using **global word–word co-occurrence statistics** across an entire corpus.

Rather than relying only on local prediction tasks:

- A word–word co-occurrence matrix is constructed
- Embeddings are learnt by optimizing the ratios of co-occurrence probabilities

This approach allows GloVe to encode semantic similarity and linear relationships and analogical structure.

3D Semantic Feature Space

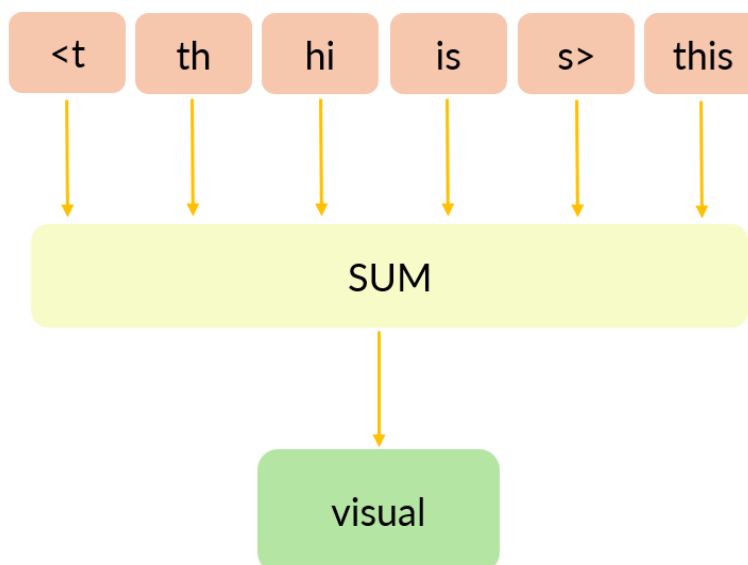


FastText

FastText extends Word2Vec by representing words as collections of character n-grams rather than indivisible tokens.

Each word is:

- Decomposed into overlapping character n-grams
- Represented as the sum of its n-gram embedding vectors



This design enables FastText to:

- Represent rare and unseen words
- Capture internal word structure such as prefixes and suffixes
- Perform well for morphologically rich languages

Word Embedding Methods: Comparison

Feature	Word2Vec	GloVe	FastText
Learning task	Predictive (Local context)	Count-based (Global matrix)	Predictive (Subword n-grams)
Key advantage	Efficient on large data sets	Captures global word patterns	Handles rare and out-of-vocabulary words

Handling out-of-vocabulary words	Cannot represent unseen words	Cannot represent unseen words	Better (via subwords)
Unit of learning	Whole word	Whole word	Character n-grams

Word Embeddings: Limitations

Word embeddings are an effective way to capture semantic similarity between words. However, they assign one fixed vector per word, which introduces several important limitations when modelling natural language.

Context-Agnostic Representations

In traditional word embedding models, each word is associated with a single vector representation regardless of how the word is used in different sentences.

As a result:

- The same word receives the same embedding in all contexts
- Contextual meaning is averaged across all usages

For example, a word with multiple meanings is represented by a single vector that blends these meanings.

Inability to Distinguish Grammatical and Entity Roles

Word embeddings do not adapt to sentence structure or syntactic roles. A word's representation remains unchanged regardless of whether it functions as a noun, verb or another entity.

This means the model cannot:

- Differentiate between grammatical roles

- Distinguish between common nouns and named entities
- Adapt representations based on sentence structure

Limited Context and Long-Range Dependencies

Word embedding models are trained using small, fixed-context windows. Only surrounding words influence the learning process.

Consequently:

- Relationships between distant words are not captured
- Paragraph-level and discourse-level meaning is ignored
- Long-range dependencies cannot be explicitly modelled

Failure With Implicit and Sarcastic Meaning

Word embeddings focus on word-level semantic associations. Sentence-level meaning is not modelled explicitly.

This leads to difficulties when:

- Meaning is implied rather than stated directly
- Sarcasm or irony reverses apparent sentiment
- Polarity depends on broader sentence interpretation

Limitations: Summary

Limitation	Core Issue	Impact on Language Understanding

Context-agnostic vectors	One fixed vector per word	Cannot adapt meaning based on usage
No grammatical awareness	Ignores syntactic and entity roles	Cannot distinguish roles or categories
Local context only	Small context windows	Misses long-range dependencies
Word-level semantics	No sentence-level modelling	Fails with implicit or sarcastic meaning

These limitations highlight a fundamental gap between word-level representations and sentence-level understanding. To model meaning effectively, representations must adapt dynamically based on the surrounding context and sentence structure.

This motivates the shift towards context-aware models, which aim to generate different representations for the same word depending on how it is used.

Need for Contextual Modelling

The limitations of traditional word embeddings arise primarily because they operate at the word level rather than the sentence level. However, natural language meaning is not determined by words in isolation but by how words interact within a sentence and across longer spans of text.

In natural language, the meaning of a word often depends on the following:

- The words surrounding it
- Its grammatical role in the sentence
- The broader sentence or discourse context

A single fixed embedding cannot reflect these variations. As a result, models based solely on word embeddings struggle to capture nuanced meaning, especially when interpretation depends on context.

To understand language more effectively, models must be able to:

- Assign different representations to the same word in different contexts
- Capture relationships between words that are far apart in a sentence
- Model sentence-level and sequence-level meaning

This requires representations that are context-dependent rather than static.

From Words to Sequences

Contextual modelling treats language as a sequence rather than a collection of independent tokens. Each word is interpreted in relation to other words in the sequence, allowing its representation to change dynamically based on context.

This shift leads to sequence modelling architectures, where:

- Inputs are processed in order
- Dependencies between tokens are explicitly modelled
- Sentence-level representations can be learnt

Recurrent Models for Language

Recurrent models were among the earliest neural architectures designed to handle sequential data. They introduced mechanisms that allow models to process inputs one step at a time while retaining information from earlier steps in the sequence.

Language is inherently sequential. A word's meaning often depends on the words following and preceding it and, sometimes, on words that appear later in the sentence.

Sequence modelling aims to:

- Process the inputs of variable length
- Preserve order information
- Capture dependencies between tokens in a sentence

Recurrent models were developed to explicitly model these properties.

A recurrent model processes a sequence token by token. At each step, the model updates an internal hidden state that serves as a summary of the sequence processed so far.

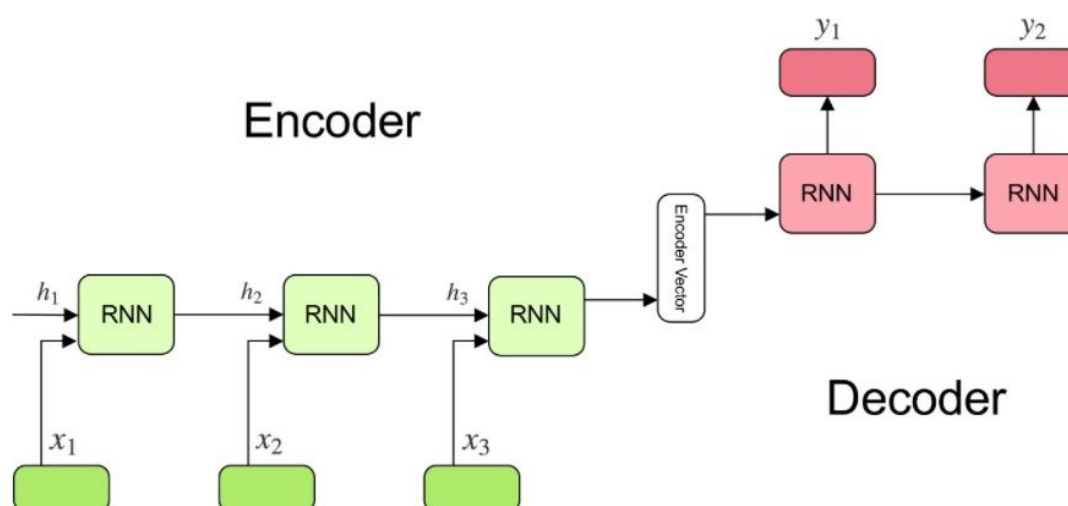
The key characteristics of recurrent models are as follows:

- The same set of parameters is reused at every time step.
- Information flows forward through the hidden state.
- Later representations depend on earlier inputs.

This design allows recurrent models to handle sequences of arbitrary length.

Encoder–Decoder Architecture

The encoder–decoder architecture is a recurrent framework designed for sequence-to-sequence tasks, where an input sequence must be transformed into an output sequence of a potentially different length.



Encoder: Processing the Input Sequence

The encoder reads the input sequence completely, processing one token at a time.

At each time step:

- The current input token is converted into a vector representation
- The encoder updates its hidden state using the current input and the previous hidden state

As the encoder progresses through the sequence:

- Early hidden states capture local, word-level information
- Later hidden states accumulate broader sentence-level context

After the final input token is processed, the encoder produces a final hidden state, which serves as a compressed representation of the entire input sequence.

This representation is expected to encode:

- The overall meaning of the sentence
- Relationships between words
- Information necessary for generating the output sequence

Decoder: Generating the Output Sequence

The decoder generates the output sequence one token at a time using the representation produced by the encoder.

At each decoding step:

- The decoder receives the encoder's final hidden state as contextual information
- It also takes as input the tokens generated so far
- Based on this information, it predicts the next output token

Once a token is generated:

- It is fed back into the decoder as input for the next step

- The process continues until a special end-of-sequence token is produced

This step-by-step generation makes the decoder an autoregressive model.

Information Flow Between an Encoder and a Decoder

The encoder and decoder communicate through a fixed-length context vector, derived from the encoder's final hidden state.

- The encoder compresses the entire input sequence into this single vector.
- The decoder relies exclusively on this vector to generate the output sequence.

This design allows the model to handle variable-length inputs and outputs but also introduces a structural bottleneck.

Encoder–Decoder Models: Applications

Recurrent encoder–decoder architectures have been used in several language tasks, including the following:

- Machine translation
- Text summarisation
- Language generation
- Speech recognition

They marked an important step towards modelling sentence-level meaning.

Recurrent Models: Limitations

Despite their conceptual strengths, recurrent models have significant practical limitations. These are listed in this table.

Aspect	Limitation	Effect
Sequential computation	Tokens processed one at a time	Slow training and inference
Fixed-length context	Entire input compressed into one vector	Information loss for long sequences
Long-range dependencies	Information passes through many steps	Difficulty retaining distant context
Parallelism	Limited parallel execution	Poor scalability

These limitations become more severe as sequence length increases. The inefficiency of sequential processing and the fixed-length context bottleneck motivated the development of attention mechanisms.

Attention Mechanism in Encoder–Decoder Models

The attention mechanism was introduced to address a key limitation of the standard encoder–decoder architecture: the need to compress an entire input sequence into a single fixed-length context vector.

Attention enhances the encoder–decoder framework without eliminating recurrence. The encoder and decoder still operate sequentially.

In the standard encoder–decoder setup:

- The encoder produces one final context representation
- The decoder relies entirely on this representation at every time step

This creates a bottleneck for long or information-dense sentences. Important details from earlier parts of the input may be lost by the time decoding begins.

Attention addresses this by allowing the decoder to access the complete sequence of encoder states rather than a single compressed vector.

Attention allows a model to:

- Access different parts of the input sequence dynamically
- Avoid compressing all information into a single vector
- Improve handling of long-range dependencies

Encoder Representations With Attention

With attention, the encoder no longer produces only one output. Instead, it generates a sequence of hidden states, one for each input token.

Each encoder hidden state:

- Encodes information about a specific input token
- Incorporates contextual information from neighbouring tokens

These hidden states collectively represent the input sequence at multiple positions.

Decoder With Attention

The decoder generates the output sequence one token at a time. However, at each decoding step, it dynamically determines which parts of the input sequence are most relevant.

Instead of relying on a fixed context vector:

- The decoder computes a new context vector at every time step
- This context vector is a weighted combination of all the encoder hidden states

The weights reflect how much attention the decoder pays to each input position when generating the current output token.

Alignment and Attention Weights

For each decoder step:

- A relevance score is computed between the current decoder state and each encoder hidden state
- These scores are normalised to form attention weights

The attention weights indicate:

- Which input tokens are most relevant
- How strongly each token contributes to the current prediction

This process is often described as learning an alignment between input and output tokens.

Context Vector in Attention-Based Models

The context vector is computed as a weighted sum of encoder hidden states.

The key differences between the standard encoder–decoder model’s vector and the context vector are as follows:

- The context vector is dynamic, not fixed.
- In the context vector, different output tokens attend to different input tokens.
- Long input sequences can be handled more effectively by the context vector.

The decoder combines this context vector with its internal state to predict the next output token.

Training With Attention

Training follows the same sequence-to-sequence objective as standard encoder-decoder model.

For each training example:

- The encoder processes the full input sequence
- The decoder predicts output tokens sequentially
- At each step, attention weights are learnt jointly with model parameters

The loss function accumulates prediction errors across all output time steps.

Attention improves encoder–decoder models by removing the fixed-length context bottleneck, allowing dynamic access to the entire input sequence and improving performance on long and complex sentences.

Attention-Based Encoder–Decoder Models: Limitations

The introduction of attention significantly improves the standard encoder–decoder architecture by removing the fixed-length context bottleneck. However, attention alone does not eliminate all structural and computational limitations of recurrent sequence models.

Continued Sequential Computation

Even with attention, the encoder and decoder remain recurrent.

This implies the following:

- Tokens are processed one time step at a time.
- Each step depends on the result of the previous step.
- Computation cannot be fully parallelised.

Consequently, training and inference remain slow, especially for long sequences.

Inefficiency for Long Sequences

In attention-based encoder–decoder models:

- The encoder produces a hidden state for every input token
- At each decoding step, attention is computed over all encoder states

For long input sequences, this leads to the following:

- Increased computational cost per decoding step
- Higher memory usage
- Slower training and inference

Although attention improves information access, it does not scale efficiently with sequence length.

Decoder Still Operates Autoregressively

The decoder generates output tokens one at a time, even with attention.

This indicates the following:

- Each output token must be generated before the next one can be processed.
- Parallel generation of output tokens is not possible.
- Latency increases with an increase in the output sequence length.

This sequential dependency limits throughput and makes real-time or large-scale generation expensive.

Limited Modelling of Global Dependencies

While attention allows the decoder to focus on relevant encoder states, the encoder itself still processes inputs sequentially.

Consequences include the following:

- Long-range dependencies must still propagate through recurrent states.
- Gradient flow over long sequences remains challenging.
- Representation learning is constrained by recurrence depth.

Attention mitigates but does not fully resolve these issues.

Limitations: Summary

Aspect	Limitation	Impact
Encoder computation	Sequential recurrence	No parallel processing
Decoder generation	Autoregressive decoding	High inference latency
Attention computation	Repeated over all encoder states	Poor scalability for long inputs
Dependency modelling	Still reliant on recurrence	Difficulty in capturing long-range context efficiently

Although attention enhances sequence-to-sequence models, the core bottleneck of recurrence remains. As data sets grow larger and sequences become longer, these limitations become increasingly restrictive.

To overcome them, a new architectural approach is required; one that:

- Retains attention as the core mechanism
- Removes recurrence entirely
- Enables parallel computation over sequences

This motivation leads to the next stage in the evolution of language models.

Transformers for Language Modelling

Transformers are neural architectures designed to model sequential language data without using recurrence. They rely entirely on attention mechanisms to capture relationships between tokens in a sequence.

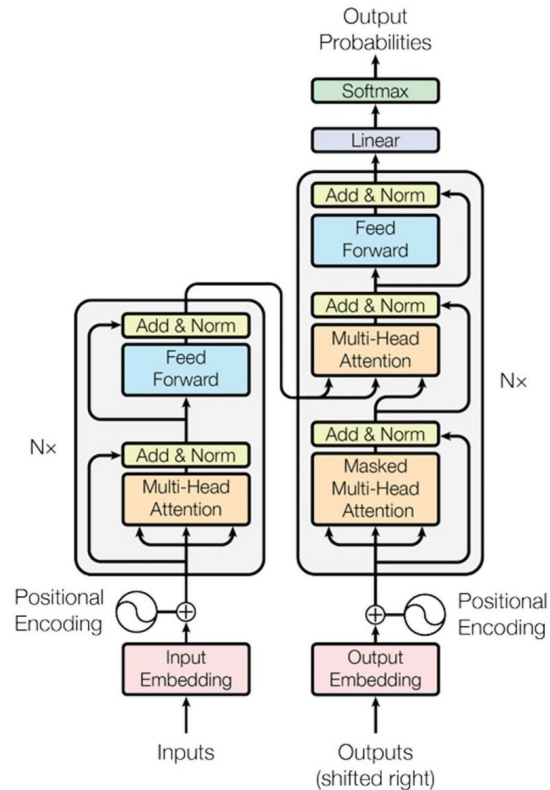
The Transformer architecture addresses the key limitations of recurrent and attention-based encoder–decoder models by removing sequential dependence in computation, enabling parallel processing of entire sequences and modelling long-range dependencies directly.

High-Level Architecture

A Transformer consists of these two main components:

- An **encoder stack**, which processes the input sequence
- A **decoder stack**, which generates the output sequence

Each component is built by stacking multiple identical layers. The sequence modelling within these layers is performed using attention mechanisms and feedforward networks.



Input Representation

Before entering the Transformer, each input token is converted into a vector representation.

This representation consists of the following:

- A **token embedding**, representing the identity of the word
- A **positional encoding**, representing the position of the word in the sequence

The two vectors are combined to form the input to the first encoder layer.

Positional Encoding

Transformers do not process tokens sequentially; so, they have no inherent notion of word order.

Positional encoding is introduced to inject information about the position of each token in the sequence.

The process involves the following steps:

- Each position in the sequence is associated with a positional vector.
- This vector is added to the token embedding.
- The combined representation encodes *what* the word is and *where* it occurs.

Positional encoding allows the model to distinguish between different word orders even though computation is parallel.

Self-Attention Mechanism

Self-attention is the core operation of the Transformer. In self-attention:

- Each token in the sequence compares itself with every other token
- The model learns how much attention to pay to other tokens when computing a representation

As a result, the representation of a token is influenced by nearby words, distant words and any other token deemed relevant by the model. This enables explicit modelling of long-range dependencies.

Encoder Stack: Working in Detail

Each encoder layer consists of two main sub-components.

Self-Attention Layer

The self-attention layer allows every token in the input sequence to attend to all other tokens.

This produces the following:

- Context-aware representations for each token
- Representations that reflect the full input sequence

Feedforward Network

After self-attention, each token representation is passed through a position-wise feedforward network.

The Key properties of the feedforward network are as follows:

- The same feedforward network is applied independently to each token.
- It introduces non-linearity.
- It refines token representations after attention.

The feedforward network does not mix information across tokens; all interactions occur in the attention layer.

Encoder Output

After passing through all encoder layers:

- Each input token is represented by a contextual embedding
- These embeddings capture information from the entire input sequence

The encoder output serves as the final representations for understanding tasks, or
The Context for the decoder in generation tasks

Decoder Stack: Working in Detail

The decoder generates the output sequence one token at a time but still uses attention-based layers.
Each decoder layer contains the following three main sub-components.

Masked Self-Attention

In the decoder's self-attention layer:

- Each token can attend only to previous tokens

- Attention to future tokens is masked

This ensures that predictions depend only on the already generated tokens and preserves the autoregressive nature of generation.

Encoder–Decoder Attention

The decoder also attends to the encoder output. This mechanism:

- Allows the decoder to focus on the relevant parts of the input sequence
- Uses encoder representations as context
- Enables input–output alignment

This is similar to attention in recurrent encoder–decoder models, but is implemented without recurrence.

Feedforward Network

As in the encoder, a feedforward network is applied independently to each token representation to refine it further.

Transformers: Key Characteristics

Aspect	Transformer Design
Recurrence	Completely removed
Core operation	Self-attention
Input processing	Fully parallel
Long-range dependencies	Explicitly modelled

Sequence order	Handled via positional encoding
----------------	---------------------------------

Transformer Design: Outcome

By removing recurrence and relying entirely on attention:

- Transformers scale efficiently to long sequences
- Training can be parallelised across tokens
- Representations capture global context more effectively

This architecture forms the foundation for multiple Transformer-based models used in modern language processing.

Variants of Transformer Architectures

Different natural language processing tasks require different patterns of information access. Some tasks require understanding the entire input at once, whereas others require generating text step by step or mapping one sequence to another.

Therefore, a single Transformer configuration is specialised into three architectural variants based on the following:

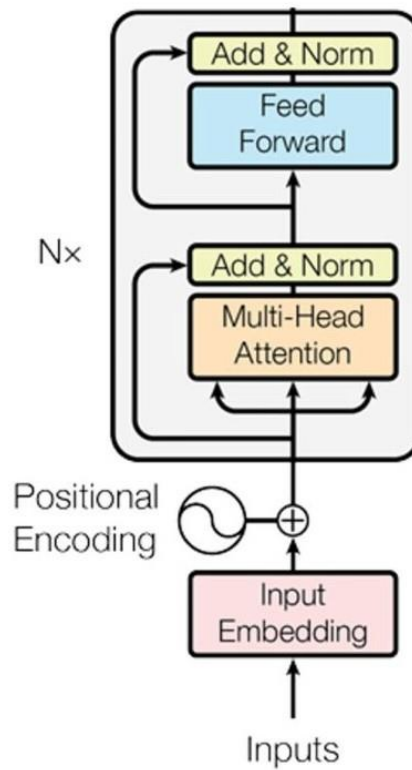
- How attention is applied
- How outputs are produced

Transformer Variants: Overview

Variant	Primary Purpose	Core Attention Pattern	Output Form
Encoder-only	Language understanding	Bidirectional self-attention	Contextual token embeddings
Decoder-only	Language generation	Causal (masked) self-attention	Generated text
Encoder–Decoder	Sequence transformation	Self-attention + cross-attention	Generated text conditioned on input

Encoder-Only Transformers

Encoder-only Transformers consist entirely of stacked Transformer encoder layers. They are designed for tasks that require understanding and representing input text, rather than generating new sequences.



Attention Behaviour

Encoder-only models use bidirectional self-attention. This means the following:

- Each token attends to all other tokens in the input sequence.
- Both left and right contexts are available simultaneously.
- No attention masking is applied.

As a result, token representations are refined using complete sentence-level context at every layer.

Output Representations

The output of an encoder-only Transformer is one contextual embedding per input token. These embeddings encode the following:

- Semantic meaning of words
- Grammatical roles
- Context-dependent relationships

No autoregressive generation mechanism is involved here. For downstream tasks, these embeddings are passed to task-specific heads such as classifiers or similarity scorers.

Training Objective: Masked Language Modelling (MLM)

Encoder-only Transformers are trained using Masked Language Modelling.

Core idea: The model learns how to predict missing words in a sentence using the surrounding context.

How MLM Works

- A subset of input tokens is masked or replaced.
- The model processes the full sequence simultaneously.
- For each masked position, the model predicts the original token.
- Predictions use the left and right context.

MLM encourages learning deep bidirectional representations, making encoder-only models highly effective for understanding tasks.

Representative Models and Families

Model	Architectural Type	Key Characteristic
BERT	Encoder-only	MLM + Next Sentence Prediction
RoBERTa	Encoder-only	MLM only, larger training corpus
DistilBERT	Encoder-only	Compressed, faster inference
ALBERT	Encoder-only	Parameter sharing for efficiency

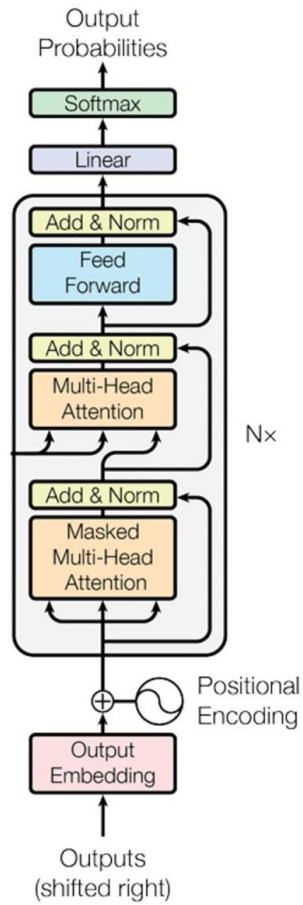
Typical Use Cases

Encoder-only Transformers are commonly used for the following:

- Text classification
- Named entity recognition
- Semantic similarity
- Extractive question answering

Decoder-Only Transformers

Decoder-only Transformers consist entirely of stacked Transformer decoder layers. They are designed for autoregressive language generation.



Attention Behaviour

Decoder-only models use causal (unidirectional) self-attention. This means:

- Each token attends only to tokens that appear before it
- Attention to future tokens is masked
- Text is generated from left to right

This ensures that the model never accesses future information during training or inference.

Output Representations

The output of a decoder-only Transformer is a probability distribution over the vocabulary for the next token. Text generation proceeds sequentially.

- The model predicts the next token.
- The predicted token is appended to the input.
- The process repeats until generation stops.

This makes decoder-only models inherently generative.

Training Objective: Causal Language Modelling (CLM)

Decoder-only Transformers are trained using Causal Language Modelling.

Core idea: The model learns how to predict the next token given all previous tokens.

How CLM Works

- For a sequence $t_1, t_2, \dots, t_n$, the model predicts t_i using $t_1 \dots t_{i-1}$
- A causal mask prevents access to future tokens
- The loss is accumulated over all next-token predictions

Training and inference follow the same left-to-right procedure, which makes CLM well-suited for text generation.

Representative Models and Families

Model	Architectural Type	Capability Focus
GPT-1	Decoder-only	Foundational generative model

GPT-2	Decoder-only	Coherent long-form text
GPT-3	Decoder-only	Few-shot learning
Later GPT variants	Decoder-only	Instructions-following

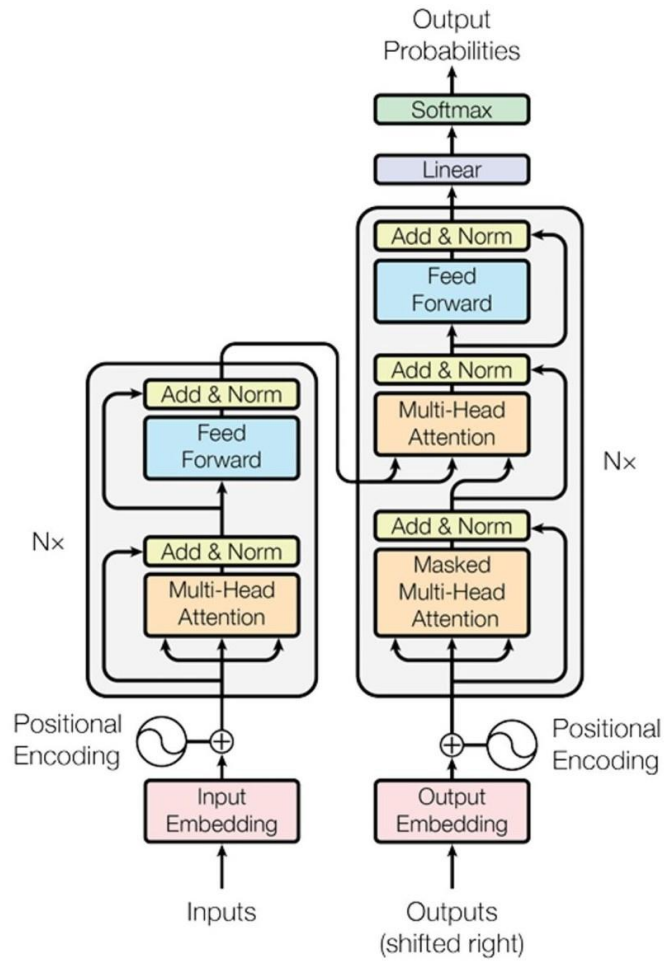
Typical Use Cases

Decoder-only Transformers are used for:

- Open-ended text generation
- Dialogue systems
- Creative writing
- Instruction-based tasks

Encoder–Decoder Transformers

Encoder–decoder Transformers combine both encoder and decoder stacks into a single architecture. They are designed for tasks that require explicit transformation from an input sequence to an output sequence.



Attention Behaviour

Encoder-decoder models use these three attention mechanisms:

1. Encoder Self-Attention
 - Bidirectional
 - Builds contextual representations of the input
2. Decoder Masked Self-Attention
 - Causal
 - Controls autoregressive generation

3. Encoder–Decoder (Cross) Attention

- The decoder attends to encoder outputs
- Enables alignment between input and output sequences

Output Representations

The output of an encoder–decoder Transformer is a generated output sequence, where each output token is conditioned on previously generated tokens and the full encoded input sequence. This produces input-grounded and controlled generation.

Training Objective: Sequence-to-Sequence Learning

Encoder–decoder Transformers are trained using sequence-to-sequence objectives.

A commonly used approach is span corruption, which has been described below:

- Contiguous spans of tokens are masked in the input.
- The encoder processes the corrupted input.
- The decoder learns how to generate the missing spans.

This objective encourages learning the following:

- Long-range dependencies
- Structured text generation
- Explicit input–output mapping

Representative Models and Families

Model	Architectural Type	Key Idea
-------	--------------------	----------

T5	Encoder–decoder	Text-to-text formulation
T5-small / base / large	Encoder–decoder	Scale variation
FLAN-T5	Encoder–decoder	Instruction tuning

Comparative Summary of Transformer Variants

Aspect	Encoder-Only (BERT)	Decoder-Only (GPT)	Encoder–Decoder (T5)
Attention behaviour	Bidirectional self-attention	Causal self-attention	Self-attention + cross-attention
Output representation	Contextual token embeddings	Next-token probabilities	Input-conditioned output tokens
Text generation	Not supported	Primary capability	Supported
Training objective	MLM	CLM	Seq2Seq (span corruption)
Task focus	Understanding	Generation	Transformation

Key Takeaways

Transformer variants differ in design intent, not in strength.

- Encoder-only models focus on understanding.
- Decoder-only models focus on generation.
- Encoder–decoder models focus on transformation.

The choice of variants should always be guided by the task requirements.

Sentence Transformers

Transformer encoders such as BERT produce contextual embeddings for individual tokens, not for entire sentences. Although these token-level representations are effective, many applications require a single fixed-size vector that represents the meaning of a full sentence.

Sentence Transformers are designed to address this requirement.

Token-Level Embeddings: Limitations

In standard encoder-only Transformers:

- Each token is represented by its own contextual embedding
- There is no explicit sentence-level representation
- Comparing sentences requires additional processing

A naïve approach is to compare sentences token by token; however, this is:

- Computationally expensive
- Difficult to scale to large collections
- Inefficient for retrieval and similarity tasks

For tasks such as semantic search or sentence similarity, it is preferable to represent each sentence using one vector.

Sentence-Level Representation Requirement

A sentence-level representation should:

- Capture the overall semantic meaning of the sentence

- Be comparable using simple similarity measures
- Scale efficiently to large data sets

Sentence Transformers fulfil these requirements by modifying how Transformer encoders are used and trained, rather than changing their internal architecture.

Sentence Transformers: Core Idea

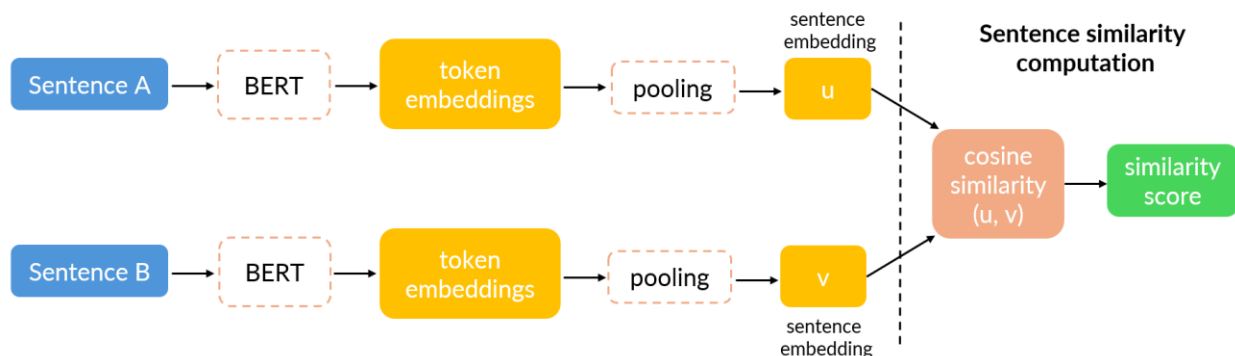
Sentence Transformers adapt Transformer encoders to produce one fixed-size embedding per sentence. This is achieved by:

- Encoding the sentence using a Transformer encoder
- Aggregating token embeddings into a single sentence embedding
- Training the model so that semantically similar sentences are closer in the embedding space

The underlying encoder remains a standard Transformer encoder.

Architecture: Bi-Encoder (Siamese) Structure

Sentence Transformers use a **bi-encoder (Siamese) architecture**.



In this setup:

- Two identical Transformer encoders are used

- Both encoders share the same parameters
- Each sentence is encoded independently

This allows sentence embeddings to be computed separately and compared efficiently.

Attention Behaviour

Sentence Transformers inherit their attention behaviour from the underlying Transformer encoder.

The key characteristics of Sentence Transformers are as follows:

- Bidirectional self-attention is used.
- Each token attends to all other tokens in the sentence.
- Full sentence context is available during encoding.

No cross-attention is involved, as sentences are processed independently.

Sentence Embeddings: Pooling Strategies

After encoding a sentence, token-level embeddings must be combined into a single vector. This is achieved using pooling strategies. Common pooling methods are given in this table.

Pooling Method	Description	Notes
Mean pooling	Average of all token embeddings	Most commonly used
Max pooling	Maximum value across tokens (dimension-wise)	Highlights dominant features

CLS pooling	Uses the embedding of the special [CLS] token	Depends on the pre-training setup
-------------	---	-----------------------------------

Mean pooling is widely preferred because it produces stable and general-purpose sentence representations.

Output Representations

The output of a Sentence Transformer is one fixed-size sentence embedding per input sentence.

These embeddings:

- Capture sentence-level semantics
- Are directly comparable using vector similarity measures
- Can be precomputed and stored for large corpora

Similarity Computation

Once sentence embeddings are obtained, similarity between sentences is computed using simple vector operations. Common similarity measures include cosine similarity and dot product.

Since embeddings are fixed-size vectors, similarity computation is fast and scalable.

Training Objective for Sentence Transformers

Sentence Transformers are trained using similarity-based objectives rather than token prediction.

Typical Training Setup

- Pairs or triplets of sentences are provided

- The model learns how to:
 - Bring semantically similar sentences closer
 - Push dissimilar sentences farther apart

This directly aligns training with downstream similarity tasks.

Comparison: Sentence Transformers vs Standard Encoder Models

Aspect	Standard Encoder	Sentence Transformer
Output	Token-level embeddings	Sentence-level embedding
Sentence comparison	Indirect and expensive	Direct and efficient
Architecture	Single encoder	Bi-encoder (shared weights)
Scalability	Limited for retrieval	Highly scalable
Typical use	Classification, tagging	Similarity, retrieval

Sentence Transformers: Applications

Sentence Transformers are commonly used for the following:

- Semantic search
- Question-answer matching
- Document and sentence clustering
- Duplicate and paraphrase detection

In these tasks, performance depends on the embedding quality and retrieval efficiency, which Sentence Transformers are designed to optimise.

Key Takeaways

Sentence Transformers extend encoder-only Transformers by:

- Producing a single embedding per sentence
- Enabling efficient similarity computation
- Scaling to large text collections

They are particularly effective when semantic comparison, rather than generation or transformation, is the primary objective.

When to Use Which Transformer Family

Requirement / Constraint	Encoder-Only Transformers (BERT Family)	Decoder-Only Transformers (GPT Family)	Encoder–Decoder Transformers (T5 Family)	Sentence Transformers (SBERT Family)
Primary goal	Language understanding	Language generation	Sequence transformation	Semantic comparison
Typical task type	Classification, tagging, analysis	Open-ended text generation	Input–output mapping	Similarity, retrieval
Input–output relationship	Input only	Prompt → continuation	Input → output	Sentence ↔ sentence
Attention behaviour	Bidirectional self-attention	Causal self-attention	Self-attention + cross-attention	Bidirectional self-attention

Access to future context	Yes	No	Encoder: Yes, Decoder: No	Yes
Output representation	Contextual token embeddings	Next-token probabilities	Generated output sequence	Single sentence embedding
Text generation capability	Not supported	Core capability	Supported	Not supported
Training objective	Masked Language Modelling (MLM)	Causal Language Modelling (CLM)	Sequence-to-sequence (e.g. span corruption)	Similarity-based objectives
Parallel processing	Fully parallel	Parallel in training, sequential in decoding	Parallel in encoding, sequential in decoding	Fully parallel
Inference latency	Low	High for long outputs	Moderate to high	Very low
Scalability to large corpora	Moderate	Moderate	Moderate	High
Example models	BERT, RoBERTa, ALBERT	GPT family	T5, FLAN-T5	Sentence-BERT, SBERT variants
When to choose	When understanding matters more than generation	When fluent text generation is required	When structured transformation is needed	When fast semantic comparison is required

Practical Interpretation

- Use encoder-only models when the task requires understanding text but not generating it.
- Use decoder-only models when the task requires generating text from a prompt.

- Use encoder–decoder models when the task requires transforming one piece of text into another.
- Use sentence transformers when the task requires comparing sentences or retrieving semantically similar text efficiently.